

Publicly Auditable Secure Multi-Party Computation

Carsten Baum*, Ivan Damgård**, and Claudio Orlandi***

Aarhus University, Denmark
{cbaum,ivan,orlandi}@cs.au.dk

Abstract. In the last few years the efficiency of secure multi-party computation (MPC) increased in several orders of magnitudes. However, this alone might not be enough if we want MPC protocols to be used in practice. A crucial property that is needed in many applications is that everyone can check that a given (secure) computation was performed correctly – even in the extreme case where all the parties involved in the computation are corrupted, and even if the party who wants to verify the result was not participating. This is especially relevant in the *clients-servers* setting, where many clients provide input to a secure computation performed by a few servers. An obvious example of this is electronic voting, but also in many types of auctions one may want independent verification of the result. Traditionally, this is achieved by using non-interactive zero-knowledge proofs during the computation.

A recent trend in MPC protocols is to have a more expensive preprocessing phase followed by a very efficient online phase, e.g., the recent so-called SPDZ protocol by Damgård et al. Applications such as voting and some auctions are perfect use-case for these protocols, as the parties usually know well in advance when the computation will take place, and using those protocols allows us to use only cheap information-theoretic primitives in the actual computation. Unfortunately no protocol of the SPDZ type supports an audit phase.

In this paper, we show how to achieve efficient MPC with a public audit. We formalize the concept of *publicly auditable secure computation* and provide an enhanced version of the SPDZ protocol where, even if all the servers are corrupted, anyone with access to the transcript of the protocol can check that the output is indeed correct. Most importantly, we do so without significantly compromising the performance of SPDZ i.e. our online phase has complexity approximately twice that of SPDZ.

Keywords: Efficient Multi-Party Computation, Public Verifiability, Electronic Voting.

* Partially supported by the European Research Commission Starting Grant 279447.

** Supported by the Danish National Research Foundation, the National Science Foundation of China (under the grant 61061130540) and also from the CFEM research center within which part of this work was performed.

*** Supported by The Danish Council for Independent Research (DFF).

1 Introduction

During the last few years MPC has evolved from a purely theoretical to a more practical tool. Several recent protocols (e.g. BeDOZa [6], TinyOT [24] and the celebrated SPDZ [14,12]) achieve incredible performance for the actual function evaluation, even if all but one player is actively corrupted. This is done by pushing all the expensive cryptographic work into an offline phase and using only simple arithmetic operations during the online phase¹. Since these protocols allow the evaluation of an arbitrary circuit over a finite field or ring, one can in particular use these protocols to implement, for instance, a *shuffle-and-decrypt* operation for a voting application or the function that computes the winning bid in an auction. It is often the case that we know well in advance the time at which a computation is to take place, and in any such case, the aforementioned protocols offer very good performance. In fact the computational work per player in the SPDZ protocol is comparable to the work one has to perform to compute the desired function in the clear, with no security.

However, efficiency is not always enough: if the result we compute securely has large economic or political consequences, such as in voting or auction applications, it may be required that correctness of the result can be verified later. Ideally, we would want that this can be done even if all parties involved in the computation are corrupted, and even if the party who wants to verify the result was not involved in the computation.

The traditional solution to this is to ask every player to commit to all his secret data and to prove in zero-knowledge for every message he sends, that this message was indeed computed according to the protocol. If a common reference string is available, we can use non-interactive zero-knowledge proofs, which allow anyone to verify the proofs and hence the result at any later time. However, this adds a very significant computational overhead, and would lead to a horribly inefficient protocol, compared to the online phase of SPDZ, for instance.

It is therefore natural to ask whether it is possible to achieve the best of both worlds and have *highly efficient MPC protocols with a high-speed online phase that are auditable*, in the sense that everyone who has access to the transcripts of the protocol can check if the result is correct *even when all the servers are corrupted*. In this paper we answer this question in the affirmative.

1.1 Contributions and Technical Overview

The model. In this work we will focus on client-server MPC protocols, where a set of parties (called the input parties) provide inputs to the actual working parties, who run the MPC protocol among themselves and make the output

¹ The offline phase is independent from the inputs and the circuit to be computed – only an upper bound on the number of multiplication gates is needed.

public². We will focus on the setting of MPC protocols for dishonest majority (and static corruptions): as long as there is one honest party we can guarantee privacy of the inputs and correctness of the results, but we can neither guarantee termination nor fairness. We will enhance the standard network model with a *bulletin board* functionality. Parties are allowed to exchange messages privately, but our protocol will instruct them also to make part of their conversation public.

Auditable MPC. Our first contribution is to provide a formal definition of the notion of *publicly auditable MPC* as an extension of the classic formalization of secure function evaluation. We require correctness and privacy when there is at least one honest party, and in addition ask that anyone, having only access to the transcript of the computation published on the bulletin board, can check the correctness of the output. This is formalized by introducing an extra, non-corruptible party (the *auditor*) who can ask the functionality if the output was correct or not³. We stress that the auditor does not need to be involved (or even exist!) before and during the protocol. The role of the auditor is simply to check, once the computation is over, whether the output was computed correctly or not.⁴

SPDZ recap. Given the motivation of this work, we are only interested in the notion of auditable MPC if it can be achieved efficiently. Therefore our starting point is one of the most efficient MPC protocols for arithmetic circuits with a cheap, information-theoretic online phase, namely SPDZ.

In a nutshell SPDZ works as follows: At the end of the offline phase all parties hold additive shares of multiplicative triples (x, y, z) with $z = x \cdot y$. Now the players can use these preprocessed triples to perform multiplications using only linear operations over the finite field (plus some interaction). Moreover, these linear operations can now be performed locally and are therefore essentially for free. However an adversary could send the honest parties a share that is different from what he received at the end of the offline phase. To make sure this is not the case, SPDZ adds information-theoretic MACs of the form $\gamma = \alpha \cdot x$ to each shared value x , where both the MAC γ and the key α are shared among the parties. These MACs are trivially linear and can therefore *follow the computation*. Once the output is reconstructed, the MAC keys are also revealed and the MACs checked for correctness, and in the case the check goes through, the honest parties accept the output.

² Note that the sets need not be distinct, and using standard transformations we can make sure that the servers do not learn the inputs nor the output of the computation (think of the inputs/output being encrypted or secret shared).

³ Of course, this only holds in the case where the computation did not abort.

⁴ In terms of feasibility, auditable MPC can be achieved by compiling a strong semi-honest protocol with NIZKs – a semi-honest MPC protocol alone would not suffice as we cannot force the parties to sample uniform randomness, nor can we trust them to force each other to do so by secure coin-tossing when everyone is corrupted. However, this would not lead to a very practical solution.

Auditable SPDZ. In order to make SPDZ auditable, we enhance each shared value x with a Pedersen commitment $g^x h^r$ to x with randomness r . The commitment key (g, h) comes from a common reference string (CRS), such that even if all parties are corrupted, those commitments are still (computationally) binding. To allow the parties to open their commitments, we provide them also with a sharing of the randomness r (each party already knows a share of x). It is easy to see that this new *representation* of values is still linear and is therefore compatible with the existing SPDZ framework. During the computation phase, the parties ignore the commitments (they are created during the offline phase, and only the openings must be sent to $\mathcal{F}_{\text{BULLETIN}}$) and it will be the job of the auditor to use the linear properties of the commitments to verify that each step of the computation was carried out correctly. Clearly the *offline phase* of SPDZ needs to be modified, in order to produce the commitments to be used by the auditor. Moreover, we have to make this preprocessing step auditable as well. We refer to the full version for more details.

1.2 An Example Application: Low-latency Voting from MPC

Our work can be seen as a part of a recent trend in understanding how generic MPC protocols perform (in terms of efficiency) in comparison to special-purpose protocols (see [11,19] for a discussion on private-set intersection). A notable example of *special purpose* secure computation protocols are mixed-networks (mix-nets), first introduced by Chaum in 1981 [9]. Here we show how our publicly auditable version of SPDZ compares favorably with mix-nets in terms of latency.

In mix-nets a number of clients submit their encrypted inputs to some servers, who jointly shuffle and decrypt the inputs in such a way that no one should be able to link the input ciphertexts with the output plaintexts, if at least one of the shuffling servers is honest. Mix-nets are of prime importance in electronic voting (like e.g. the famous *Helios* [1] system). A disadvantage of mix-nets is that they are *inherently sequential*: server i cannot start shuffling before receiving the output of the shuffle performed by server $i-1$. Now, given that the voter's privacy depends on the assumption that there is at least 1 out of n uncorrupted server, it is desirable to increase the number of parties involved in the shuffle as much as possible. However, when using mix-nets the latency of the protocol is linear in n , and therefore increasing n has a very negative impact on the total efficiency of the protocol, here measured by the time between the last voter casts his vote and the output of the election is announced. We argue here that implementing a shuffle using a generic protocol like SPDZ makes the latency independent of the number of servers performing the shuffle.

More formally, let n be the number of servers, m the number of input ciphertexts, λ a computational security parameter and sec a statistical security parameter. The *computational latency* of mix-nets, here defined as the time we have to wait before all servers have done their computational work, will be at

least $O(n \cdot m \cdot \lambda^3)$.⁵ Using SPDZ, the computational latency is $O(m \cdot \log(m) \cdot \text{sec}^2)$,⁶ since the *total* complexity of SPDZ is linear in n and the servers work in parallel (this was even verified experimentally). Therefore mix-nets are more expensive by a factor of $(\frac{n}{\log m} \cdot \frac{\lambda^3}{\text{sec}^2})$: This is a significant speedup when n grows – note also that typical values of λ for public-key cryptography can be one or two orders of magnitudes greater than typical values for a statistical security parameter sec (only field operations are performed during the SPDZ online phase). Clearly, to verify the impact in practice one would have to implement both approaches and compare them. We leave this as an interesting future work.

The above comparison only considers the efficiency of the two approaches. However, as argued before, in applications like voting it is crucial to allow the voters to check that the outcome of the election is correct. Most mix-nets protocol already achieve public verifiability using non-interactive zero-knowledge proofs for the correctness of shuffles. This motivates our study of auditable generic protocols.

1.3 Related Work

For certain applications, there already exist *auditable protocols*. The idea is known in the context of e.g. electronic voting as *public verifiability*, and can also be found concerning online auctions and secret sharing. To the best of our knowledge, the term *public verifiability* was first used by Cohen and Fischer in [10]. Widely known publicly auditable voting protocols are those of Schoenmakers [28] and Chaum et al. [8] and the practical Helios [1]. Also stronger notions for voting protocols have been studied, see e.g. [27,22,30]. Verifiability also appeared for secret sharing schemes [28,17,29] and auctions [23,26]. We refer the reader to the mentioned papers and the references therein for more information on these subjects. It is crucial to point out that our suggested approach is not just another voting protocol – instead we lift verifiability to arbitrary secure computations. In this setting, the notion of public verifiability has not been studied, with the exception of [15], where the author presents a general transformation that turns *universally satisfiable* protocols into instances that are *auditable* in our sense. This transformation is general and slows down the computational phase of protocols, whereas our approach is tailored for fast computations.

In publicly verifiable delegation of computation (see e.g. [18,16] and references therein) a computationally limited device delegates a computation to the cloud and wants to check that the result is correct. Verifiable delegation is useless unless verification is more efficient than the evaluation. Note that in some sense our requirement is the opposite: We want our workers to work as little as possible, while we are fine with asking the auditor to perform more expensive computation.

External parties have been used before in cryptography to achieve otherwise impossible goals like fairness [20], but note that in our case *anyone can be the*

⁵ The λ^3 factor is there because of the re-randomization step that is crucially done in every mix-net. Using “onions” of encryptions would not be more efficient.

⁶ The $m \cdot \log m$ factor comes from the optimal shuffle of Ajtai et al. [2].

auditor and does not need to be online while the protocol is executed. This is a qualitative difference with most of the other semi-trusted parties that appear in the literature. A recent work [3] investigated an enhanced notion of covert security, that allows anyone to determine if a party cheated or not given the transcript of the protocol – note that the goal of our notion is different, as we are interested in what happens when *all* parties are corrupted.

2 Defining Auditable MPC

In this section, we formalize our notion of publicly auditable MPC. We add a new party to the standard formalization which *only performs the auditing* and does not need to participate during the offline or the online phase. This auditor does not even have to exist when the protocol is executed, but he can check the correctness of a result based on a protocol transcript. This *formal hack* makes it possible to guarantee correctness even if everyone participating in the computation is corrupted⁷.

As mentioned, we put ourselves in the client-server model, so the parties involved in an auditable MPC protocols are:

The input parties: We consider m parties $\mathcal{I}_1, \dots, \mathcal{I}_m$ with inputs $(x_1, \dots, x_m)$.

The computing parties: We consider n parties $\mathcal{P}_1, \dots, \mathcal{P}_n$ that participate in the computation phase. Given a set of inputs $x_1, \dots, x_m$ they compute an output $y = C(x_1, \dots, x_m)$ for some circuit C over a finite field. Note that $\{\mathcal{I}_1, \dots, \mathcal{I}_m\}$ and $\{\mathcal{P}_1, \dots, \mathcal{P}_n\}$ might not be distinct.

The auditor: After the protocol is executed, anyone acting as the auditor $\mathcal{T}_{\text{AUDIT}}$ can retrieve the transcript of the protocol τ from the bulletin board and (using only the circuit C and the output y) determine if the result is valid or not.

Our security notion is the standard one if there is at least one honest party (i.e. we guarantee privacy, correctness, etc.). However standard security notions do not give any guarantee in the *fully malicious* setting, i.e. when all parties are corrupted. We tweak the standard notions slightly and ask an additional property, called *auditable correctness*.

This notion captures the fact that in the fully malicious case, the input cannot be kept secret from the adversary $\mathcal{A}$. But we still want to prove that if the computing parties deviate from the protocol, this will be caught by $\mathcal{T}_{\text{AUDIT}}$, who has access to the transcript of the execution using a bulletin board $\mathcal{F}_{\text{BULLETIN}}$.

More formally, our definition for auditable correctness is as follows:

Definition 1 (Auditable Correctness). *Let C be a circuit, $x_1, \dots, x_m$ be inputs to C , y be a potential output of C and τ be a protocol transcript for the*

⁷ We are not adding a semi-trusted third party to the actual protocol: Our guarantee is that if there exist at least one honest party in the universe who cares about the output of the computation, that party can check at any time that the output is correct.

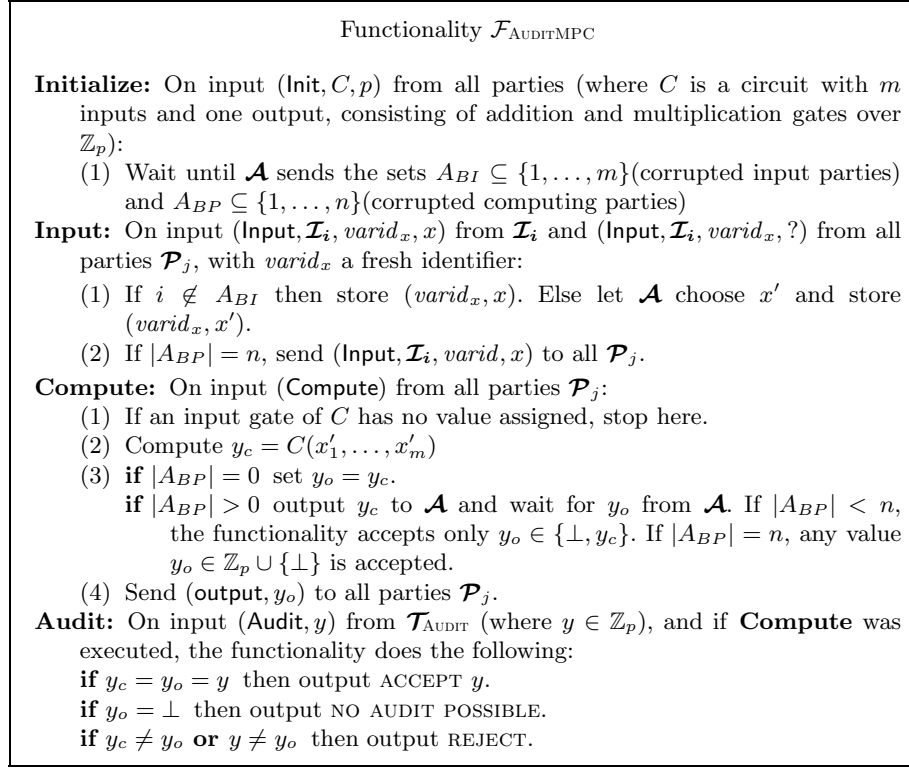


Fig. 1. The ideal functionality that describes the online phase

evaluation of the circuit C . We say that an MPC protocol π satisfies Auditable Correctness if the following holds: The auditor $\mathcal{T}_{\text{AUDIT}}$ with input τ outputs **ACCEPT** y with overwhelming probability if the circuit C on input $x_1, \dots, x_m$ produces the output y . At the same time the auditor $\mathcal{T}_{\text{AUDIT}}$ will return **REJECT** (except with negligible probability) if τ is not a transcript of an evaluation of C or if $C(x_1, \dots, x_m) \neq y$.

In Figure 1 we present an ideal functionality that formalizes our notion of auditable MPC in the UC setting (where we use the same notation as before). We use this ideal world-real world paradigm, because it simplifies the proof, whereas the game-based definition gives a better intuition about audibility. The protocol/simulator transcript can then be used as in Definition 1, and a protocol that is secure in the ideal world-real world setting is also auditable correct according to the definition.

To simplify the exposition, $\mathcal{F}_{\text{AUDITMPC}}$ is only defined for one output value y . This can easily be generalized.

Note that we only defined our $\mathcal{F}_{\text{AUDITMPC}}$ for deterministic functionalities. The reason for this is that when all parties are corrupted even the auditor cannot check whether the players followed the protocol correctly in the sense of using

real random tapes. This can be solved (using standard reductions) by letting the input parties contribute also random tapes and define the randomness used by the functionality as the XOR of those random tapes – but in the extreme case where all the input parties are corrupted this will not help us.

3 An Auditable MPC Protocol

We now present an MPC protocol that is an extension of [14,12]. We obtain a fast online phase, which almost only consists of opening shared values towards parties.

Our Setup. Let $p \in \mathbb{P}$ be a prime and G be some abelian group (in multiplicative notation) of order p where the *Discrete Logarithm Problem* (DLP) is hard to solve. The MPC protocol will evaluate a circuit C over $\mathbb{Z}_p$, whereas we use the group G to ensure auditability. Therefore, let $g, h \in G$ be two generators of the group G where h is chosen such that $\log_g(h)$ is not known (e.g. based on some CRS). For two values $x, r \in \mathbb{Z}_p$, we define $pc(x, r) := g^x h^r$.

We assume that a secure channel towards the input parties can be established, that a broadcast functionality is available and that we have access to a bulletin board $\mathcal{F}_{\text{BULLETIN}}$ (Fig. 2), a commitment functionality $\mathcal{F}_{\text{COMMIT}}$ ⁸ (Fig. 3) and a procedure to jointly produce random values $\mathcal{P}_{\text{PROVIDERANDOM}}$ (Fig. 4)⁹. To implement $\mathcal{P}_{\text{PROVIDERANDOM}}$ let $\mathcal{U}_s(q, l)$ be a random oracle with seed $s \in \{0, 1\}^*$ that outputs a uniformly random element from $\mathbb{Z}_q^l$. We use the bulletin board $\mathcal{F}_{\text{BULLETIN}}$ to keep track of all those values that are broadcasted. Observe that no information that was posted to $\mathcal{F}_{\text{BULLETIN}}$ can ever be changed or erased.

Sharing Values for the Online Phase. All computations during the online phase are done using additively-shared values. The parties are committed to each such shared value using a MAC key α and a commitment to the shared value. The key α is also additively-shared among the parties, where party $\mathcal{P}_i$ holds share α_i such that $\alpha = \sum_{i=1}^n \alpha_i$, and the commitments to each value are publicly known.

We define the $\langle \cdot \rangle$ -representation of a shared value as follows:

Definition 2. Let $r, s, e \in \mathbb{Z}_p$, then the $\langle r \rangle$ -representation of r is defined as

$$\langle r \rangle := ((r_1, \dots, r_n), (\gamma(r)_1, \dots, \gamma(r)_n))$$

where $r = \sum_{i=1}^n r_i$ and $\alpha \cdot r = \sum_{i=1}^n \gamma(r)_i$. Each player $\mathcal{P}_i$ will hold his shares $r_i, \gamma(r)_i$ of such a representation. Moreover, we define

⁸ This other commitment functionality might be implemented by a hash function/random oracle, and is used whenever the linear operations of the commitment scheme are not necessary.

⁹ Note that the random oracle model and $\mathcal{F}_{\text{COMMIT}}$ were already assumptions used in the original SPDZ protocol, our extra assumptions are the existence of $\mathcal{F}_{\text{BULLETIN}}$ and the DLP-hard group G .

The ideal functionality $\mathcal{F}_{\text{BULLETIN}}$	
Store:	On input $(\text{Store}, id, i, msg)$ from $\mathcal{P}_i$, where id was not assigned yet, the functionality stores (id, i, msg) .
Reveal IDs:	On input (All) from party $\mathcal{P}_i$ the functionality reveals all assigned id -values to $\mathcal{P}_i$
Reveal message:	On input (Getmsg, id) from $\mathcal{P}_i$, the functionality checks whether id was assigned already. If so, then it returns (id, j, msg) to $\mathcal{P}_i$. Otherwise it returns $(id, \perp, \perp)$.

Fig. 2. The ideal Functionality for the Bulletin board

$$\begin{aligned}
\langle r \rangle + \langle s \rangle &:= ((r_1 + s_1, \dots, r_n + s_n), (\gamma(r)_1 + \gamma(s)_1, \dots, \gamma(r)_n + \gamma(s)_n)) \\
e \cdot \langle r \rangle &:= ((e \cdot r_1, \dots, e \cdot r_n), (e \cdot \gamma(r)_1, \dots, e \cdot \gamma(r)_n)) \\
e + \langle r \rangle &:= ((r_1 + e, r_2, \dots, r_n), (\gamma(r)_1 + e \cdot \alpha_1, \dots, \gamma(r)_n + e \cdot \alpha_n))
\end{aligned}$$

This representation is closed under linear operations:

Remark 1. Let $r, s, e \in \mathbb{Z}_p$. We say that $\langle r \rangle \hat{=} \langle s \rangle$ if both $\langle r \rangle, \langle s \rangle$ reconstruct to the same value. Then it holds that

$$\langle r \rangle + \langle s \rangle \hat{=} \langle r + s \rangle, \quad e \cdot \langle r \rangle \hat{=} \langle e \cdot r \rangle, \quad e + \langle r \rangle \hat{=} \langle e + r \rangle$$

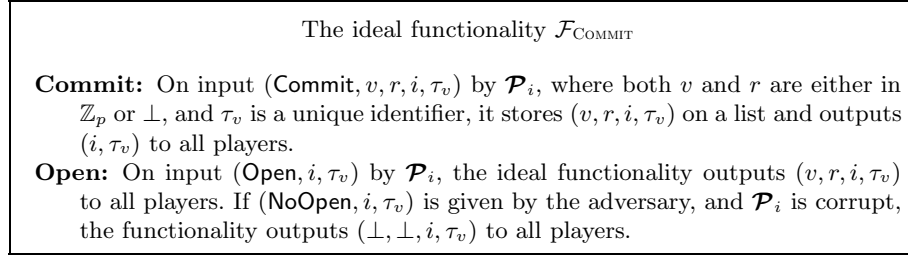
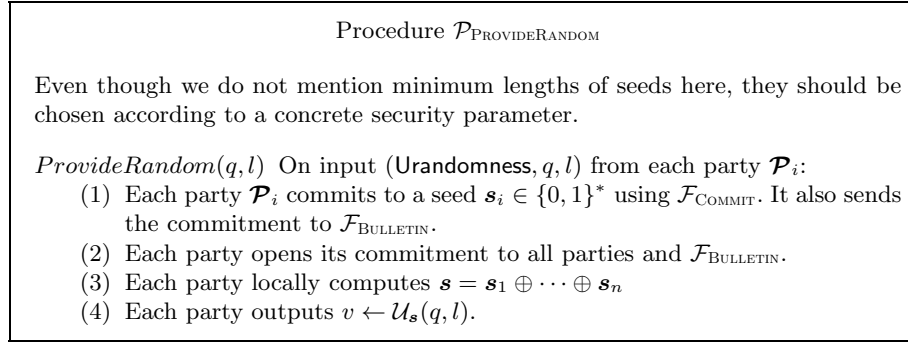
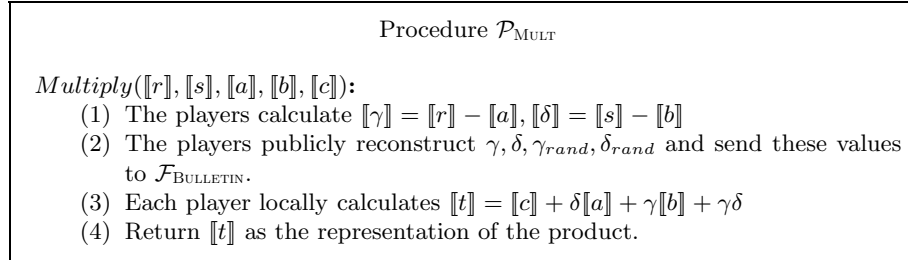
A value that is shared as above is reconstructed or *opened*¹⁰ by summing up all shares. The correctness of this opening can be checked by checking the MAC (we will use a protocol where α will not be revealed). A value $\langle a \rangle$ can either be *publicly opened* if every player $\mathcal{P}_i$ broadcasts its share a_i , or *opened towards* $\mathcal{P}_i$ if every other party $\mathcal{P}_j, j \neq i$ sends its share a_j to $\mathcal{P}_i$. Similarly, if the players *open towards* $\mathcal{F}_{\text{BULLETIN}}$ this means that they send their shares of the particular value to the bulletin board.

During the online phase, the parties either open sharings (without revealing the MACs) or do the linear operations defined above. Together with the Beaver circuit randomization technique from [4] and a MAC checking procedure for the output phase, this already yields an actively secure MPC scheme that is secure against up to $n - 1$ corrupted players¹¹.

The $\llbracket \cdot \rrbracket$ -representation. In order to make SPDZ auditable we enhance the way shared values are represented and stored. In a nutshell we force the computing parties to commit to the inputs, opened values and outputs of the computation. All intermediate steps can then be checked by performing the computation using the data on $\mathcal{F}_{\text{BULLETIN}}$. The commitment scheme is information-theoretically hiding, and we will carry both the actual value $\langle r \rangle$ as well as the randomness $\langle r_{\text{rand}} \rangle$ of the commitment through the whole computation.

¹⁰ We use both terms for it in this paper.

¹¹ Provided that the offline phase generates valid multiplication triples and random values together with MACs.

**Fig. 3.** The Ideal Functionality for Commitments**Fig. 4.** A protocol to jointly generate random values**Fig. 5.** Protocol to generate the product of two $\llbracket \cdot \rrbracket$ -shared values

The commitment to a value r will be a Pedersen commitment (see [25]) $pc(r, r_{rand})$. When we open a $\llbracket \cdot \rrbracket$ -representation, we reconstruct both r and r_{rand} .¹² This way the commitment is also opened (it is already published on $\mathcal{F}_{\text{BULLETIN}}$) and everyone can check that it is correct (but the computing parties do not need to do so).

Definition 3. Let $r, r_{rand} \in \mathbb{Z}_p$ and $g, h \in G$ where both g, h generate the group, then we define the $\llbracket r \rrbracket$ -representation for r as

$$\llbracket r \rrbracket := (\langle r \rangle, \langle r_{rand} \rangle, pc(r, r_{rand}))$$

¹² Our different flavours of opening for $\langle \cdot \rangle$ -representations can be applied here as well.

where $\langle r \rangle, \langle r_{rand} \rangle$ are shared among the players as before.

We define linear operations on the representations as before:

Definition 4. Let $a, b, a_{rand}, b_{rand}, e \in \mathbb{Z}_p$. We define the following linear operations on $\llbracket \cdot \rrbracket$ -sharings:

$$\begin{aligned}\llbracket a \rrbracket + \llbracket b \rrbracket &:= (\langle a \rangle + \langle b \rangle, \langle a_{rand} \rangle + \langle b_{rand} \rangle, pc(a, a_{rand}) \cdot pc(b, b_{rand})) \\ e \cdot \llbracket a \rrbracket &:= (e \cdot \langle a \rangle, e \cdot \langle a_{rand} \rangle, (pc(a, a_{rand}))^e) \\ e + \llbracket a \rrbracket &:= (e + \langle a \rangle, \langle a_{rand} \rangle, pc(e, 0) \cdot pc(a, a_{rand}))\end{aligned}$$

With a slight abuse in notation, we see that

Remark 2. Let $r, s, e \in \mathbb{Z}_p$. It holds that

$$\llbracket r \rrbracket + \llbracket s \rrbracket \triangleq \llbracket r + s \rrbracket, \quad e \cdot \llbracket r \rrbracket \triangleq \llbracket e \cdot r \rrbracket, \quad e + \llbracket r \rrbracket \triangleq \llbracket e + r \rrbracket$$

In order to multiply two representations, we rely on the protocol in Figure 5 (as in [4]): Let $\llbracket r \rrbracket, \llbracket s \rrbracket$ be two values where we want to calculate a representation $\llbracket t \rrbracket$ such that $t = r \cdot s$. Assume the existence of a triple $(\llbracket a \rrbracket, \llbracket b \rrbracket, \llbracket c \rrbracket)$ such that a, b are uniformly random and $c = a \cdot b$. Then one can obtain $\llbracket t \rrbracket$ using $\mathcal{P}_{MULT}$.

Most interestingly, one does not have to perform the computations on the commitments during the online phase. Instead, *only the $\langle \cdot \rangle$ -representations are manipulated!*

Shared Randomness from an Offline Phase. Our online phase relies on the availability of $\llbracket \cdot \rrbracket$ -representations of random values and multiplication triples. In Figure 6 and 8 we define the functionality $\mathcal{F}_{SETUP}$ that describes our preprocessing protocol, which is essentially an *auditable* version of the SPDZ preprocessing functionality. If all parties are corrupted, the functionality might output an incorrect result – however this can be checked by the auditor. Since we assume that g, h come from a CRS, the audit is still correct in this setting.

The Online Phase. The online phase of our protocol is presented in Figure 9. To create the transcript, every party puts all values it ever *sends* or *receives* onto $\mathcal{F}_{BULLETIN}$ (except for the private reconstruction of input values)¹³. The check of the MACs is done as in SPDZ using the protocol in Figure 11.

4 Security of the Online Phase

In this section, we will prove that for all poly-time adversaries $\mathcal{A}$ there exists a simulator $\mathcal{S}_{ONLINE}$ such that $\Pi_{AUDITMPC}$ is indistinguishable from $\mathcal{F}_{AUDITMPC}$ to every poly-time environment $\mathcal{Z}$. As we argued before, this also implies that $\Pi_{AUDITMPC}$ fulfills the *auditable correctness* requirement from Definition 1.

We start with the following Lemma from [12, Lemma 1] about correctness and soundness of the MAC check. We then prove the security of the online phase in Theorem 1.

¹³ This does not break the security, because (informally speaking) this is the same information that an $\mathcal{A}$ receives if he corrupts $n - 1$ parties.

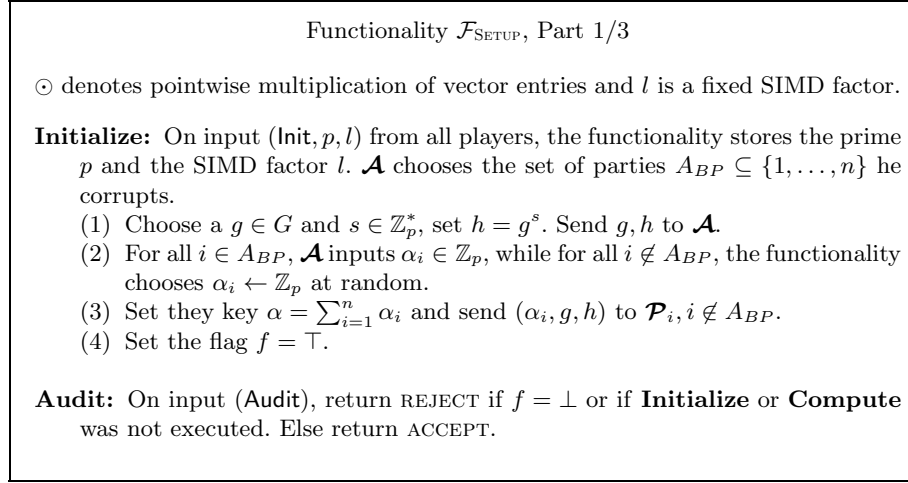


Fig. 6. The ideal functionality that describes the offline phase

Lemma 1. *Let $p \in \mathbb{P}$. On input $(a_1, \gamma(a_1), \dots, a_t, \gamma(a_t), p)$ $\mathcal{P}_{\text{CHECKMAC}}$ is correct and sound:*

- If $\forall i : \alpha \cdot a_i = \gamma(a_i)$ then it returns 1 with probability 1.*
- If $\exists i : \alpha \cdot a_i \neq \gamma(a_i)$ then it rejects except with probability $2/p$.*

Theorem 1. *In the $\mathcal{F}_{\text{SETUP}}, \mathcal{F}_{\text{BULLETIN}}, \mathcal{F}_{\text{COMMIT}}$ -hybrid model with a random oracle, the protocol Π_{AUDITMPC} implements $\mathcal{F}_{\text{AUDITMPC}}$ with computational security against any static adversary corrupting all parties if the DLP is hard in the group G .*

Proof. We prove the above statement by providing a simulator $\mathcal{S}_{\text{ONLINE}}$ (see Figure 12). The simulator is divided for two cases, for the honest minority ($\mathcal{S}_{\text{ONLINE}, \text{NORMAL}}$ in Figure 13) and the fully malicious setting ($\mathcal{S}_{\text{ONLINE}, \text{FULL}}$ in Figure 14).

At Least One Honest Party. The simulator runs an instance of Π_{AUDITMPC} with the players controlled by $\mathcal{Z}$ and simulated honest parties. For **Initialize**, **Input**, **Add**, **Multiply** it performs the same steps as in Π_{AUDITMPC} , only that it uses a fixed input 0 for the simulated honest parties during **Input**. Since every set of at most $n - 1$ shares of a value is uniformly random and does not reveal any information about the shared secret, this cannot be distinguished from a real execution.

During **Output**, we adjust the shares of one simulated honest party to agree with the correct output y from $\mathcal{F}_{\text{AUDITMPC}}$: The simulator obtained the result y' of the simulated computation, hence it can adjust the share of a simulated honest party. Moreover, it also adjusts the MAC share as depicted in $\mathcal{S}_{\text{ONLINE}, \text{NORMAL}}$ using the MAC key α provided by $\mathcal{F}_{\text{SETUP}}$. As argued in [14], the distribution of these shares of the simulated honest parties is the same as during a protocol execution.

Functionality $\mathcal{F}_{\text{SETUP}}$, Part 2/3

Compute: On input $(\text{GenerateData}, T, \rho)$ from all players with T, ρ multiples of l :

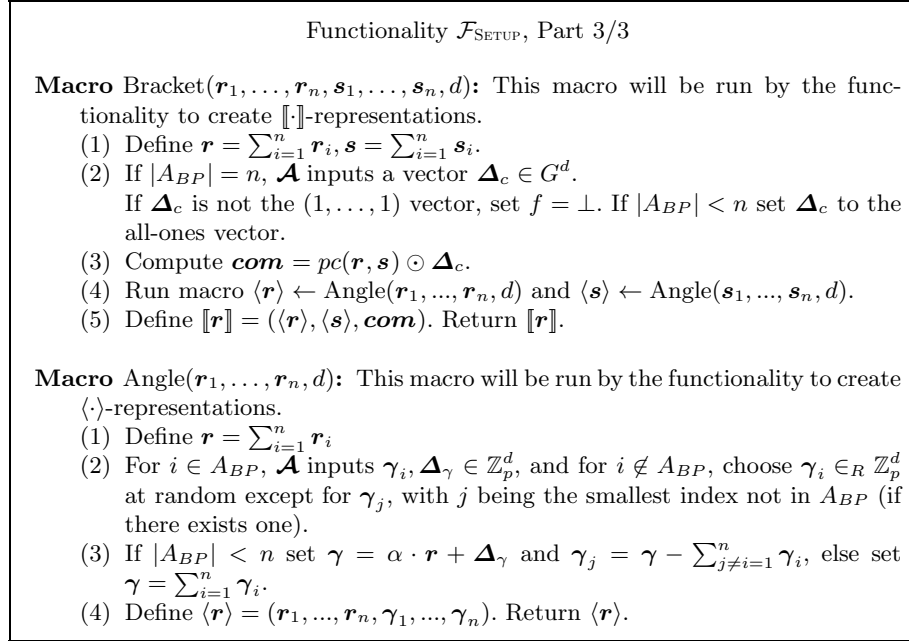
- (1) *RandomValues*(T, l):
 - (1.1) For each $i \notin A_{BP}$ choose uniformly random $\mathbf{r}_i, \mathbf{s}_i \leftarrow \mathbb{Z}_p^T$, send these to $\mathcal{P}_i$ and $pc(\mathbf{r}_i, \mathbf{s}_i)$ to $\mathcal{A}$.
 - (1.2) For $i \in A_{BP}$, $\mathcal{A}$ inputs $\mathbf{r}_i, \mathbf{s}_i \in \mathbb{Z}_p^T$.
 - (1.3) Compute $\llbracket \mathbf{r} \rrbracket \leftarrow \text{Bracket}(\mathbf{r}_1, \dots, \mathbf{r}_n, \mathbf{s}_1, \dots, \mathbf{s}_n, T)$.
 - (1.4) Send the commitments of $\llbracket \mathbf{b} \rrbracket$ to $\mathcal{F}_{\text{BULLETIN}}$.
 - (1.5) Return $(\llbracket \mathbf{r} \rrbracket)$.
- (2) *Triples*(ρ, l):
 - (2.1) For $i \notin A_{BP}$, the functionality samples $\mathbf{a}_i, \mathbf{a}_{rand,i}, \mathbf{b}_i, \mathbf{b}_{rand,i} \in \mathbb{Z}_p^\rho$ at random, sends them to $\mathcal{P}_i$ and $pc(\mathbf{a}_i, \mathbf{a}_{rand,i}), pc(\mathbf{b}_i, \mathbf{b}_{rand,i})$ to $\mathcal{A}$.
 - (2.2) For $i \in A_{BP}$, $\mathcal{A}$ inputs $\mathbf{a}_i, \mathbf{a}_{rand,i}, \mathbf{b}_i, \mathbf{b}_{rand,i} \in \mathbb{Z}_p^\rho$.
 - (2.3) For each $i \notin A_{BP}$ sample uniformly random $\mathbf{o}_i \in G^\rho$ and send them to $\mathcal{A}$.
 - (2.4) For $i \in A_{BP}$ let $\mathcal{A}$ choose $\mathbf{c}_i, \mathbf{c}_{rand,i} \in \mathbb{Z}_p^\rho$.
 - (2.5) Define $\mathbf{a} = \sum_{j=1}^n \mathbf{a}_j, \mathbf{b} = \sum_{j=1}^n \mathbf{b}_j$.
 - (2.6) Let $j \notin A_{BP}$ be the smallest index of an honest player (if any). For all $i \notin A_{BP}, i \neq j$ choose $\mathbf{c}_i \in \mathbb{Z}_p^\rho$ uniformly at random and $\mathbf{c}_{rand,i} \in \mathbb{Z}_p^\rho$ subject to the constraint that $\mathbf{o}_i = pc(\mathbf{c}_i, \mathbf{c}_{rand,i})$ using s . For $\mathcal{P}_j$ let $\mathbf{c}_j = \mathbf{a} \odot \mathbf{b} - \sum_{i \in [n], i \neq j} \mathbf{c}_i$ and $\mathbf{c}_{rand,j} \in \mathbb{Z}_p^\rho$ such that $\mathbf{o}_j = pc(\mathbf{c}_j, \mathbf{c}_{rand,j})$ using s . Send $\mathbf{c}_i, \mathbf{c}_{rand,i}$ to $\mathcal{P}_i$.
 - (2.7) Let $\mathbf{c} = \sum_{i=1}^n \mathbf{c}_i$.
 - (2.8) Run the macros $\llbracket \mathbf{a} \rrbracket \leftarrow \text{Bracket}(\mathbf{a}_1, \dots, \mathbf{a}_n, \mathbf{a}_{rand,1}, \dots, \mathbf{a}_{rand,n}, \rho)$, $\llbracket \mathbf{b} \rrbracket \leftarrow \text{Bracket}(\mathbf{b}_1, \dots, \mathbf{b}_n, \mathbf{b}_{rand,1}, \dots, \mathbf{b}_{rand,n}, \rho)$, $\llbracket \mathbf{c} \rrbracket \leftarrow \text{Bracket}(\mathbf{c}_1, \dots, \mathbf{c}_n, \mathbf{c}_{rand,1}, \dots, \mathbf{c}_{rand,n}, \rho)$.
 - (2.9) Let $L' = \{1, \dots, \rho\}$.
 - (2.10) If $|A_{BP}| = n$ then let $\mathcal{A}$ input L , otherwise let $L = L'$. If $L \neq L'$ then set $f = \perp$.
 - (2.11) Send the commitments of $(\llbracket \mathbf{a}[m] \rrbracket, \llbracket \mathbf{b}[m] \rrbracket, \llbracket \mathbf{c}[m] \rrbracket)_{m \in L}$ to $\mathcal{F}_{\text{BULLETIN}}$.
 - (2.12) Return $(\llbracket \mathbf{a}[m] \rrbracket, \llbracket \mathbf{b}[m] \rrbracket, \llbracket \mathbf{c}[m] \rrbracket)_{m \in L}$.

Fig. 7. The ideal functionality that describes the offline phase, continued

The commitment is information-theoretically hiding, and since the discrete logarithm $\log_g(h)$ is known to $\mathcal{S}_{\text{ONLINE,NORMAL}}$, it can compute a randomness value y_{rand} that correctly opens the commitment in $\llbracket y \rrbracket$ as posted on $\mathcal{F}_{\text{BULLETIN}}$ for the value y instead of y' . It then adjusts a share of y'_{rand} such that (y', y'_{rand}) open the commitment. Once again, the distribution of this share of y_{rand}' agrees with the distribution in a real protocol execution.

If moreover $\mathcal{Z}$ decides to stop the execution, then $\mathcal{S}_{\text{ONLINE,NORMAL}}$ will forward this to the ideal functionality and $\mathcal{Z}$ will not receive any additional information, as in the real execution.

During the **Audit** phase, we also do exactly the same as in the protocol. Note that both **Output** and **Audit** will always reveal the correct values from

**Fig. 8.** The ideal functionality that describes the output of the offline phase, continued

$\mathcal{F}_{\text{AUDITMPC}}$ in the simulated case. We have to show that in the real protocol, the probability that $\mathcal{A}$ can cheat is negligible.

Output: There are three ways how the output can be incorrect with respect to the inputs and the calculated function, which is if a multiplication triple was not correct even though it passed the check, or if a dishonest party successfully adjusted the MACs during the computation, or it successfully cheated during the output phase. As argued in [14], the first event only happens with probability $1/p$. If $\mathcal{A}$ can adjust the MACs correctly with non-negligible probability, then it can guess the secret MAC key α – which contradicts that it only holds at most $n-1$ shares of it which reveal no information. For the third case, Lemma 1 implies that this can only happen with probability $2/p$. Since we set p to be exponential in the security parameter, the distributions are statistically indistinguishable.

Audit: We focus on the two cases when $\mathcal{F}_{\text{AUDITMPC}}$ and Π_{AUDITMPC} disagree about the output of **Audit**. The conditions under which $\mathcal{S}_{\text{OFFLINE,NORMAL}}$ and Π_{AUDITMPC} output NO AUDIT POSSIBLE are the same.

- (1) $\mathcal{F}_{\text{AUDITMPC}}$ outputs ACCEPT y when Π_{AUDITMPC} outputs REJECT does not happen due to the construction of $\mathcal{S}_{\text{OFFLINE,NORMAL}}$.
- (2) $\mathcal{F}_{\text{AUDITMPC}}$ outputs REJECT when Π_{AUDITMPC} outputs ACCEPT y . $\mathcal{A}$ replaced the output with $\perp$, but $\mathcal{P}_{\text{CHECKMAC}}$ passed successfully. This happens with probability at most $2/p$ according to Lemma 1.

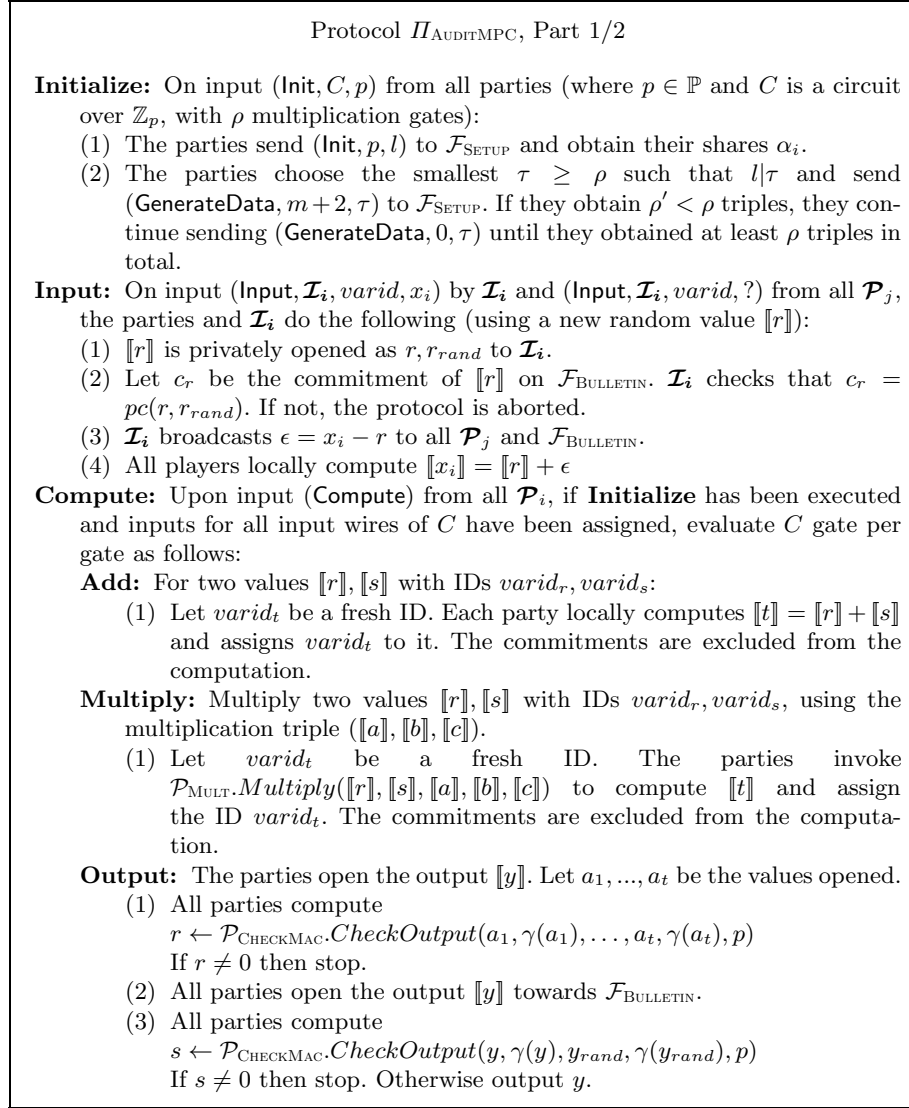
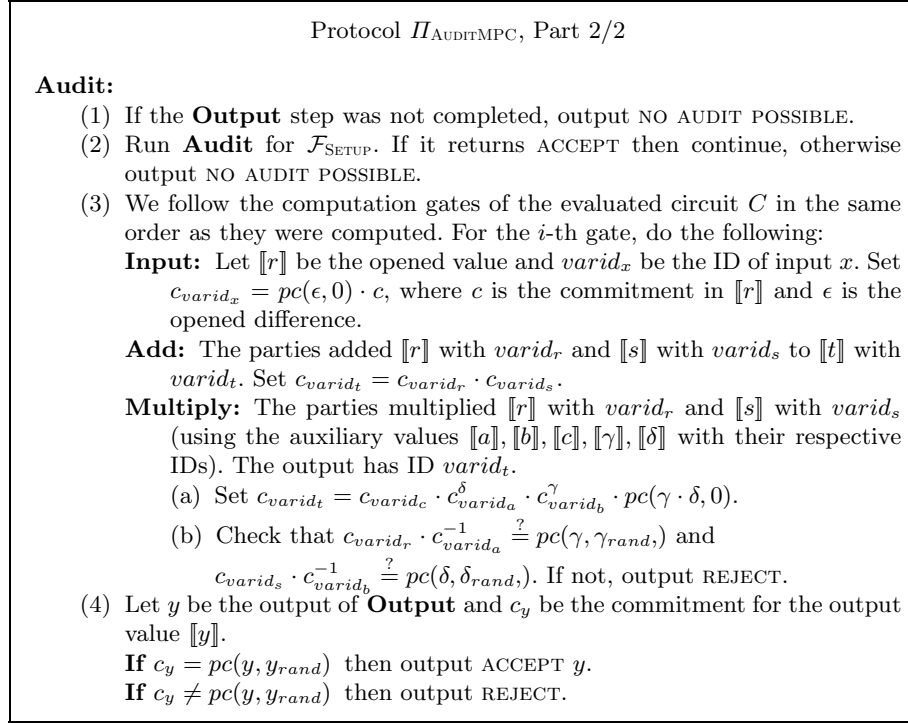
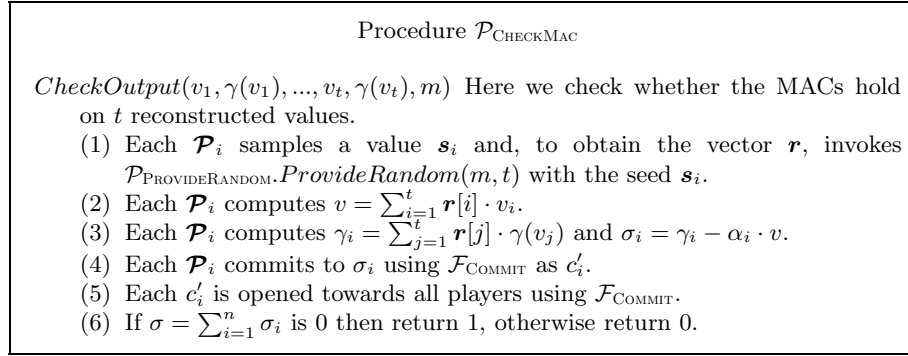


Fig. 9. The protocol for the online phase

Fully Malicious Setting. The intuition behind $\mathcal{S}_{\text{ONLINE, FULL}}$ is that we let $\mathcal{Z}$ send arbitrary messages during the online phase. But since all messages for $\mathcal{F}_{\text{SETUP}}$ go through $\mathcal{S}_{\text{ONLINE, FULL}}$, we extract the used inputs after the fact which we then can use with $\mathcal{F}_{\text{AUDITMPC}}$. Observe that, since we cannot guarantee privacy, no inputs must be substituted. During the **Audit**, we run the protocol of Π_{AuditMPC} also in the simulator (but with different outputs, as we shall see). The difference between both again is the output of **Audit** in both worlds.

**Fig. 10.** The protocol for the online phase, continued**Fig. 11.** Procedure to check validity of MACs

- (1) $\mathcal{F}_{\text{AUDITMPC}}$ outputs ACCEPT y when Π_{AUDITMPC} outputs REJECT does not happen due to the construction of $\mathcal{S}_{\text{OFFLINE, FULL}}$.
- (2) $\mathcal{F}_{\text{AUDITMPC}}$ outputs REJECT when Π_{AUDITMPC} outputs ACCEPT y . $\mathcal{Z}$ replaced the output with another value y' (and also y'_{rand}) that open the commitment c_y . But in step (3) of **Compute**, the simulator already obtained y such that

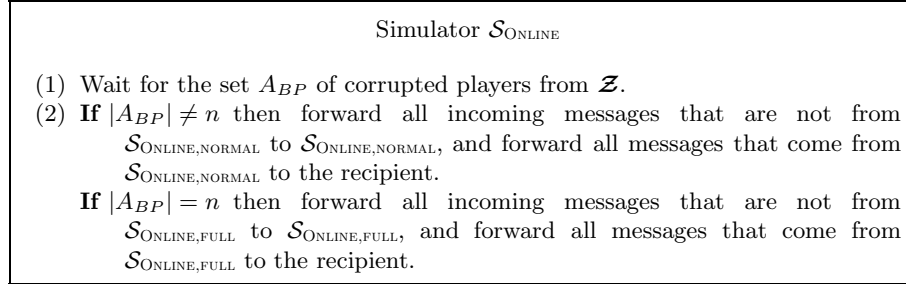


Fig. 12. Simulator for the online phase

$pc(y, y_{rand}) = pc(y', y'_{rand})$ for some y_{rand} .¹⁴ This implies a solution of the DLP in poly-time, contradicting the assumption. □

5 On the Efficiency of Our Solution

In this section, we will outline why the practical efficiency of the offline and audit phase of our protocol crucially depends on how fast commitments can be computed and checked. We will moreover present a few optimizations for these tasks.

Asymptotic Efficiency. In terms of asymptotic efficiency, our suggested on-line phase is as efficient as the SPDZ protocol. Practically, the number of local field operations and sent values increases by a modest factor of two, plus some additional work for each input-providing party (to check whether the commitment is correct). It is an interesting open problem to see if one could get rid of even this minor slowdown.

To be more precise, we have to distinguish between the field operations in $\mathbb{Z}_p$ and the group operations in G . In the standard setting, where each party provides $O(1)$ inputs and $O(1)$ output values are jointly computed, and where the number of gates in our circuit is upper-bounded by $|C|$, all operations of the online phase (**Input**, **Add**, **Multiply**, **Output**) together can be performed by each player doing at most $O(n \cdot |C|)$ field operations. Assuming that we use Pedersen commitments to implement $\mathcal{F}_{\text{COMMIT}}$ in practice, we obtain an extra $O(n \cdot \log p)$ group operations during **Input** and **Output**. In terms of network load, each party sends or receives $O(n \cdot |C|)$ field elements over the network during the **Input** phase and while the computation is carried out, and $O(n)$ elements from $\mathbb{Z}_p$ and G during **Output**.

We moreover have to discuss our new **Audit** phase of the protocol (we exclude $\mathcal{F}_{\text{SETUP}}$ from the discussion). The strategy of **Audit** is to *follow the computation with the commitments*. Here, the number of operations in $\mathbb{Z}_p$ is $O(n \cdot |C|)$, which

¹⁴ Computing y_{rand} from C and the randomization values of the inputs is straightforward. We omitted this computation here.

Simulator $\mathcal{S}_{\text{ONLINE,NORMAL}}$

The values g, h are provided as a CRS by this simulator, so $s = \log_g(h)$ is known as well as α .

Initialize: On input (Init, C, p) from $\mathcal{Z}$:

- (1) Set up $\mathcal{F}_{\text{BULLETIN}}$ and start a local instance Π of Π_{AUDITMPC} with the dishonest parties (and simulated honest parties).
- (2) Run a copy of $\mathcal{F}_{\text{SETUP}}$, with which $\mathcal{Z}$ and the simulated honest parties communicate through the simulator.
- (3) Run **Initialize** and **Compute** of $\mathcal{F}_{\text{SETUP}}$ as in Π_{AUDITMPC} .

Input: On input $(\text{Input}, \mathcal{I}_i, \text{valid}, \cdot)$ by $\mathcal{I}_i$ and $(\text{Input}, \mathcal{I}_i, \text{valid}, ?)$ from and $\mathcal{Z}$:

If $\mathcal{I}_i$ is honest then follow Π_{AUDITMPC} for a fake input 0.

If $\mathcal{I}_i$ is dishonest then extract the input value x_i from Π and send it to $\mathcal{F}_{\text{AUDITMPC}}$. Execute this step with x_i in $\mathcal{F}_{\text{AUDITMPC}}$.

Compute: Upon input **(Compute)** from $\mathcal{Z}$, if **Initialize** has been executed and inputs for all input gates of C have been provided, evaluate C gate per gate as follows:

Add: Follow the steps of **Add** in Π_{AUDITMPC} .

Multiply: Follow the steps of **Multiply** in Π_{AUDITMPC} .

Output: Obtain the output y from $\mathcal{F}_{\text{AUDITMPC}}$ and simulate Π_{AUDITMPC} as follows:

- (1) Generate correct shares for the simulated honest parties for Π :
 - (1.1) Let $\mathcal{P}_i$ be a simulated honest party and y' be the output of Π with $\mathcal{Z}$ right now. Let $\llbracket y' \rrbracket = (\langle y' \rangle, \langle y'_{\text{rand}} \rangle, c = pc(y', y'_{\text{rand}}))$, where $y' = \sum_k y'_{o,k}$ and $y'_{\text{rand}} = \sum_k y'_{\text{rand},o,k}$. For all honest $\mathcal{P}_j$ with $j \neq i$, let $y'_{q,j} = y'_{o,j}$, $y'_{\text{rand},q,j} = y'_{\text{rand},o,j}$ ^a.
 - (1.2) For $\mathcal{P}_i$ set $y'_{q,i} = y'_{o,i} + (y - y')$ and $\gamma'_{q,i} = \gamma'_{o,i} + \alpha(y - y')$. We have $s \neq 0$, so $s^{-1} \bmod p$ exists. Set $y_{\text{rand}} = (y' - y + s \cdot y'_{\text{rand}}) / s \bmod p$, and $y'_{\text{rand},q,i} = y'_{\text{rand},o,i} + (y_{\text{rand}} - y'_{\text{rand}})$, $\gamma'_{\text{rand},q,i} = \gamma'_{\text{rand},o,i} + \alpha \cdot (y_{\text{rand}} - y'_{\text{rand}})$.
 - (1.3) Let $y' = \sum_k y_{q,k}$, $y'_{\text{rand}} = \sum_k y'_{\text{rand},q,k}$, $\gamma' = \sum_k \gamma'_{q,k}$ and $\gamma'_{\text{rand}} = \sum_k \gamma'_{\text{rand},q,k}$.
- (2) Follow the protocol Π_{AUDITMPC} to check the MACs according to step 1 of **Output**. If that step fails, let $\mathcal{F}_{\text{AUDITMPC}}$ deliver $\perp$ to the honest parties and stop.
- (3) Send the shares of the simulated honest parties of the output $\llbracket y \rrbracket$ to $\mathcal{F}_{\text{BULLETIN}}$. If $\mathcal{Z}$ does not provide shares of $\llbracket y \rrbracket$ for all dishonest parties, then let $\mathcal{F}_{\text{AUDITMPC}}$ set $y' = \perp$ and stop.
- (4) Run $\mathcal{P}_{\text{CHECKMAC}}$ as in Π_{AUDITMPC} . If the MAC on the output $\llbracket y \rrbracket$ is correct, let $\mathcal{F}_{\text{AUDITMPC}}$ set $y' = y$, otherwise $y' = \perp$.

Audit: Run **Audit** as in Π_{AUDITMPC} with the malicious players. Then invoke **Audit** in $\mathcal{F}_{\text{AUDITMPC}}$ and output **REJECT** if it is the output of $\mathcal{F}_{\text{AUDITMPC}}$. If not, reveal what $\mathcal{F}_{\text{AUDITMPC}}$ outputs.

^a Similarly, the MAC keys $\gamma'_{o,j}$, $\gamma'_{\text{rand},o,j}$ of those parties are not touched.

Fig. 13. Simulator for honest minority

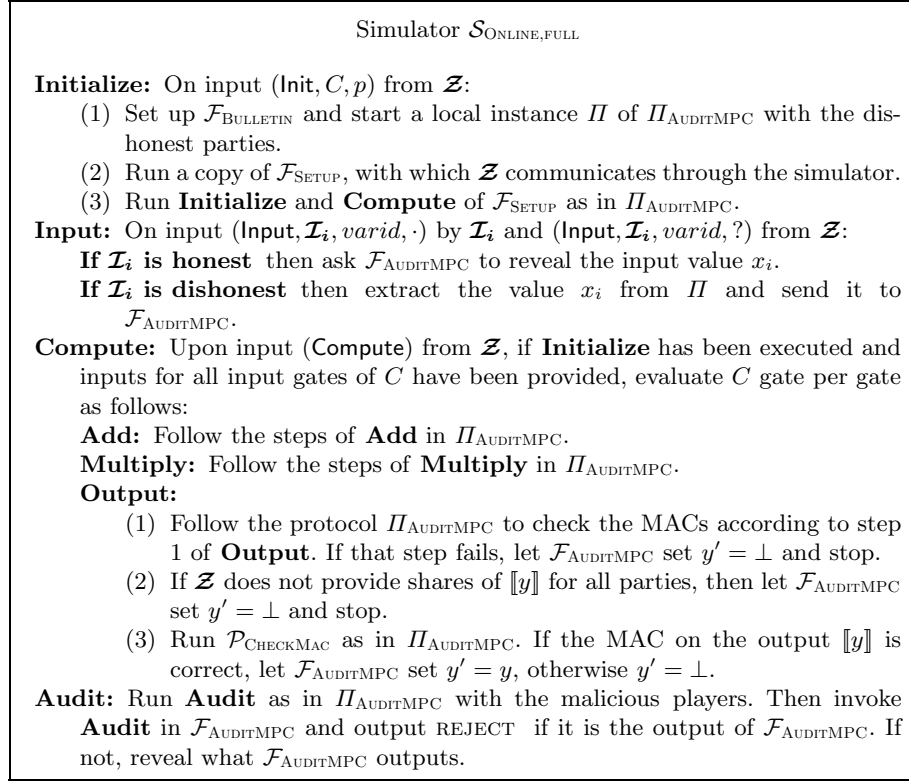


Fig. 14. Simulator for the fully malicious setting

is comparable to the online phase. In addition, the algorithm performs the gate operations on commitments and checks whether every opening of a commitment was correct – this in total requires $O((n + C) \cdot \log p)$ group operations.

To check whether the commitments are correctly opened, the audit process computes a random linear combination of the opened commitments (using coefficients from $\mathbb{Z}_p$) and the values which should open them (instead of checking all of them independently). This randomized check fails with probability $2^{-\log p}$. In practice, one would choose the coefficients of the random linear combination from the smaller interval $[0, \dots, 2^k - 1]$ (where we can have $k \ll \log p$), thus saving operations in G .

Towards a Faster Offline Phase. Though the offline phase of [14] can directly be extended to support the computation of the commitments (as we describe in the full version), one can use different optimizations for it. First of all, computing the commitments can be made faster using preprocessing as in [7, 21]. Moreover, it is possible to reduce the total number of commitments (introducing a moderate slowdown during the online phase) as follows:

Instead of computing *one commitment per value*, one can also use s pairwise distinct generators $g_1, \dots, g_s \in \mathbb{Z}_p$ together with just one randomness parameter, where generator g_i is used to commit to the i th value.

A representation $(x_1, \dots, x_t, r, g_1^{x_1} \cdots g_t^{x_t} h^r)$ of t values in parallel is componentwise linear, and multiplications can also be performed as before (now for multiple elements in parallel). We observe that the computation of a commitment with many generators can be substantially faster than computing all commitments individually. This optimization, similar to [13], works for a large class of circuits. We moreover note that, in order to use this optimization, one also has to precompute *permutations between the representations* which must then be used during the online phase. This leads to a moderate slowdown during the evaluation of the circuit.

Tweaks for the Audit Phase. The audit process, as explained in Figure 9, basically consists of (1) performing linear operations on commitments and (2) checking whether commitments open to the correct values. Whereas we see no approach to speed up the first part, we will address the second one using a well-known technique from [5].

Let $c_1, \dots, c_n \in G$ be the commitments and let $x_1, \dots, x_n, r_1, \dots, r_n$ be the values that should open them. We want to establish that $\forall i \in \{1, \dots, n\} : c_i = g^{x_i} h^{r_i}$.

The is to compute a random linear combination of all commitments, and thus check all of them at once. We choose the coefficients of the random combination from the interval $0, \dots, 2^k - 1$.

Now computing such a random linear combination will yield a false positive with probability $\approx 2^{-k}$, but we can adjust the error probability here and make it independent of the field description size $\log p$ (remember that also G has to be a DLP-hard group of order p). This also yields less computational overhead, as we only have to raise group elements to at most 2^k th powers. The algorithm looks as follows:

- (1) Choose $\mathbf{a} \leftarrow \{0, \dots, 2^k - 1\}^n$ uniformly at random.
- (2) Check that $\prod_i c_i^{a^{[i]}} = \prod_i (g^{x_i} h^{r_i})^{a^{[i]}} = g^{\sum_i a^{[i]} x_i} h^{\sum_i a^{[i]} r_i}$.

Bellare et al. show in [5] that this algorithm indeed fails to correctly verify with probability 2^{-k} . Moreover, one can use a recursive approach to gain further speedup for a large number of commitments. We refer to [5] for more details.

6 Summary and Open Problems

In this paper, we described how to formally lift MPC into a setting where all servers are malicious. We outlined how this concept can then be securely realized on top of the SPDZ protocol. Though our approach can also be implemented for other MPC protocols, we focused on SPDZ since, even as an publicly auditable scheme, its online phase is very efficient. We note that our protocol would also work for Boolean circuits, but this would introduce a significant slowdown (since the MACs must then be defined as elements of an extension field over $\mathbb{F}_2$, which

leads to a significant overhead). It is an interesting future direction to design an efficient auditable protocol optimized for Boolean circuits or circuits over fields with small characteristic.

With respect to online voting, there exist stronger degrees of auditability than we presented. An example is the notion of *universal verifiability* (see e.g. [27,22]) where the auditor must not know the output of the computation. We also do not provide *accountability* (see e.g. Küsters et al. [30]), and leave it as an open question whether similar, efficient protocols can be achieved in this setting.

We leave a working implementation of our scheme as a future work. As our protocol is very similar in structure to the original SPDZ, it should be possible to implement it easily on top of the existing codebase of [12].

Acknowledgements. We want to thank the anonymous reviewers for their helpful comments.

References

1. Helios, B.A.: Web-based open-audit voting. In: USENIX Security Symposium, vol. 17, pp. 335–348 (2008)
2. Ajtai, M., Komlós, J., Szemerédi, E.: An $o(n \log n)$ sorting network. In: STOC, pp. 1–9 (1983)
3. Asharov, G., Orlandi, C.: Calling out cheaters: Covert security with public verifiability. In: Wang, X., Sako, K. (eds.) ASIACRYPT 2012. LNCS, vol. 7658, pp. 681–698. Springer, Heidelberg (2012)
4. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Feigenbaum, J. (ed.) CRYPTO 1991. LNCS, vol. 576, pp. 420–432. Springer, Heidelberg (1992)
5. Bellare, M., Garay, J.A., Rabin, T.: Fast batch verification for modular exponentiation and digital signatures. In: Nyberg, K. (ed.) EUROCRYPT 1998. LNCS, vol. 1403, pp. 236–250. Springer, Heidelberg (1998)
6. Bendlin, R., Damgård, I., Orlandi, C., Zakarias, S.: Semi-homomorphic encryption and multiparty computation. In: Paterson, K.G. (ed.) EUROCRYPT 2011. LNCS, vol. 6632, pp. 169–188. Springer, Heidelberg (2011)
7. Brickell, E.F., Gordon, D.M., McCurley, K.S., Wilson, D.B.: Fast exponentiation with precomputation. In: Rueppel, R.A. (ed.) EUROCRYPT 1992. LNCS, vol. 658, pp. 200–207. Springer, Heidelberg (1993)
8. Chaum, D., Ryan, P.Y.A., Schneider, S.: A practical voter-verifiable election scheme. In: De Capitani di Vimercati, S., Syverson, P., Gollmann, D. (eds.) ESORICS 2005. LNCS, vol. 3679, pp. 118–139. Springer, Heidelberg (2005)
9. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM* 24(2), 84–90 (1981)
10. Cohen, J.D., Fischer, M.J.: A robust and verifiable cryptographically secure election scheme. In: FOCS, vol. 85, pp. 372–382 (1985)
11. De Cristofaro, E., Tsudik, G.: Experimenting with fast private set intersection. In: Katzenbeisser, S., Weippl, E., Camp, L.J., Volkamer, M., Reiter, M., Zhang, X. (eds.) Trust 2012. LNCS, vol. 7344, pp. 55–73. Springer, Heidelberg (2012)
12. Damgård, I., Keller, M., Larraia, E., Pastro, V., Scholl, P., Smart, N.P.: Practical covertly secure mpc for dishonest majority - or: Breaking the spdz limits. In: Crampton, J., Jajodia, S., Mayes, K. (eds.) ESORICS 2013. LNCS, vol. 8134, pp. 1–18. Springer, Heidelberg (2013)

13. Damgård, I., Zakarias, S.: Constant-overhead secure computation of boolean circuits using preprocessing. In: Sahai, A. (ed.) TCC 2013. LNCS, vol. 7785, pp. 621–641. Springer, Heidelberg (2013)
14. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 643–662. Springer, Heidelberg (2012)
15. Jacobus, S., de Hoogh, A.: Design of Large Scale Applications of Secure Multiparty Computation: Secure Linear Programming. PhD thesis, Technische Universiteit Eindhoven (2012)
16. Fiore, D., Gennaro, R.: Publicly verifiable delegation of large polynomials and matrix computations, with applications. In: ACM Conference on Computer and Communications Security, pp. 501–512 (2012)
17. Fujisaki, E., Okamoto, T.: A practical and provably secure scheme for publicly verifiable secret sharing and its applications. In: Nyberg, K. (ed.) EUROCRYPT 1998. LNCS, vol. 1403, pp. 32–46. Springer, Heidelberg (1998)
18. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct nizks without pcps. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 626–645. Springer, Heidelberg (2013)
19. Huang, Y., Evans, D., Katz, J.: Private set intersection: Are garbled circuits better than custom protocols? In: NDSS (2012)
20. Küpçü, A., Lysyanskaya, A.: Optimistic fair exchange with multiple arbiters. In: Gritzalis, D., Preneel, B., Theoharidou, M. (eds.) ESORICS 2010. LNCS, vol. 6345, pp. 488–507. Springer, Heidelberg (2010)
21. Lim, C.H., Lee, P.J.: More flexible exponentiation with precomputation. In: Desmedt, Y.G. (ed.) Advances in Cryptology - CRYPTO 1994. LNCS, vol. 839, pp. 95–107. Springer, Heidelberg (1994)
22. Moran, T., Naor, M.: Receipt-free universally-verifiable voting with everlasting privacy. In: Dwork, C. (ed.) CRYPTO 2006. LNCS, vol. 4117, pp. 373–392. Springer, Heidelberg (2006)
23. Naor, M., Pinkas, B., Sumner, R.: Privacy preserving auctions and mechanism design. In: Proceedings of the 1st ACM Conference on Electronic Commerce, pp. 129–139. ACM (1999)
24. Nielsen, J.B., Nordholt, P.S., Orlandi, C., Burra, S.S.: A new approach to practical active-secure two-party computation. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 681–700. Springer, Heidelberg (2012)
25. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) Advances in Cryptology - CRYPTO 1991. LNCS, vol. 576, pp. 129–140. Springer, Heidelberg (1992)
26. Sako, K.: An auction protocol which hides bids of losers. In: Imai, H., Zheng, Y. (eds.) PKC 2000. LNCS, vol. 1751, pp. 422–432. Springer, Heidelberg (2000)
27. Sako, K., Kilian, J.: Receipt-free mix-type voting scheme. In: Guillou, L.C., Quisquater, J.-J. (eds.) Advances in Cryptology - EUROCRYPT 1995. LNCS, vol. 921, pp. 393–403. Springer, Heidelberg (1995)
28. Schoenmakers, B.: A simple publicly verifiable secret sharing scheme and its application to electronic voting. In: Wiener, M. (ed.) CRYPTO 1999. LNCS, vol. 1666, pp. 148–164. Springer, Heidelberg (1999)
29. Stadler, M.A.: Publicly verifiable secret sharing. In: Maurer, U.M. (ed.) Advances in Cryptology - EUROCRYPT 1996. LNCS, vol. 1070, pp. 190–199. Springer, Heidelberg (1996)
30. Truderung, T., Vogt, A., Küsters, R.: Accountability: definition and relationship to verifiability. In: Proceedings of the 17th ACM Conference on Computer and Communications Security, pp. 526–535. ACM (2010)